

"A pipeline you cannot replay is not a pipeline — make every change a deployable artifact."

SUPPLEMENTARY · MODULE 11

DevOps and CI/CD

Move from commit to OpenShift with evidence: CI, image promotion, GitOps, quality gates, and the release discipline a bank expects.



DEVOPS PRINCIPLES AND CULTURE · DEVOPS LIFECYCLE FOR MICROSERVICES



CORE DEVOPS PRINCIPLES

- Collaboration across development, operations, security, and quality teams
- Automation of repeated delivery steps to reduce manual error
- Continuous feedback and improvement flowing from monitoring back into planning



COMMON MISTAKE

- Treating DevOps as only Jenkins or GitHub Actions instead of shared ownership and practice

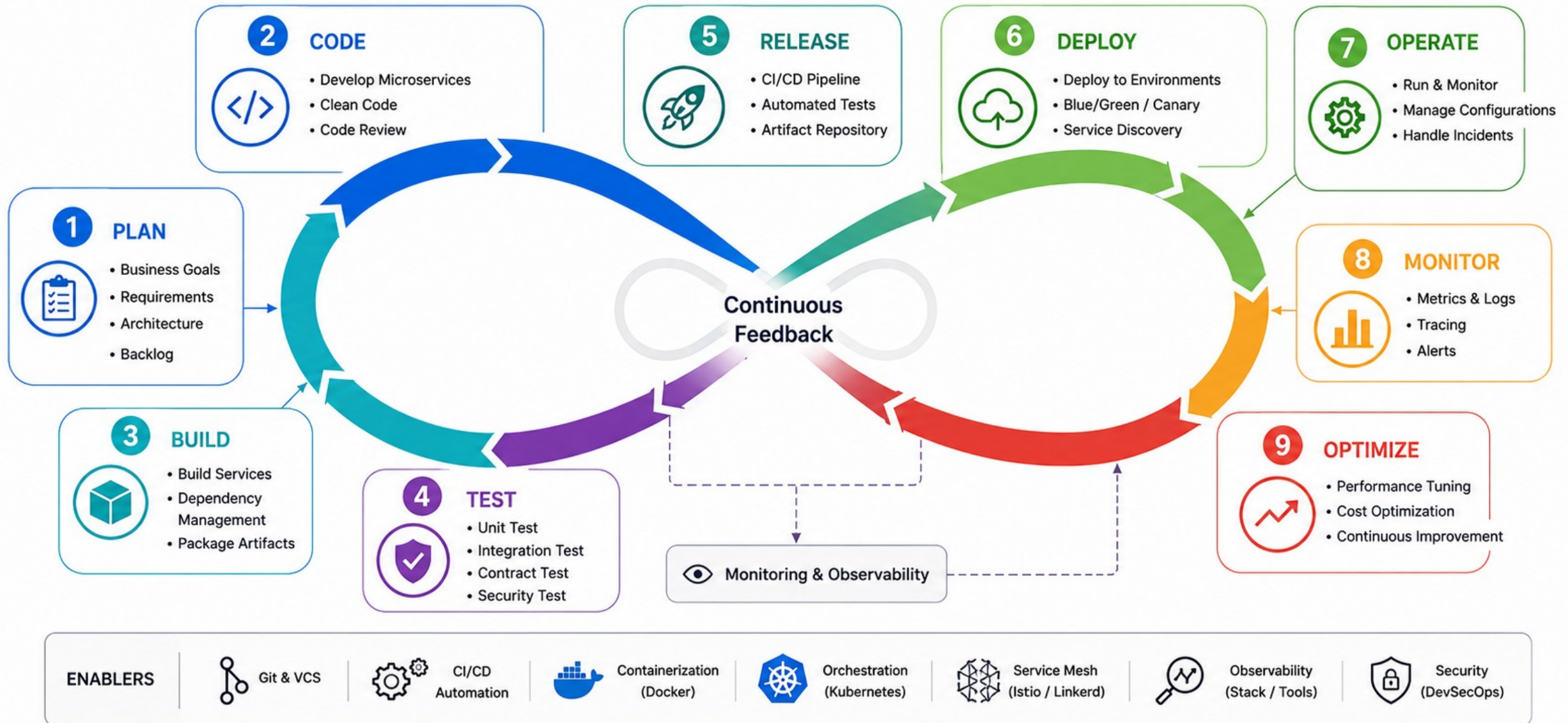


THE EIGHT LIFECYCLE STAGES

- Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor
- Feedback from monitoring and operations flows back into planning, closing the loop

KEY TAKEAWAY Collaboration, automation, and continuous feedback matter more than which specific tools a team adopts. • Account Service moves through this full loop for every feature, with monitoring data shaping the next planning cycle.

DevOps Lifecycle for Microservices



CONTINUOUS INTEGRATION · CONTINUOUS DELIVERY VS. CONTINUOUS DEPLOYMENT · PIPELINE AS CODE



WHAT CI VALIDATES

- Does the code compile? Do tests pass? Does it package successfully?
- Does code quality meet standards? Are dependencies acceptable?



THE KEY DISTINCTION

- **Continuous Delivery:** automatically build, test, and prepare for deployment -- human approval still required
- **Continuous Deployment:** automatically deploy approved changes to production after passing required checks



TYPICAL CI STAGES

- Checkout -> Compile -> Unit Tests -> Integration Tests -> Static Analysis -> Package -> Publish Reports

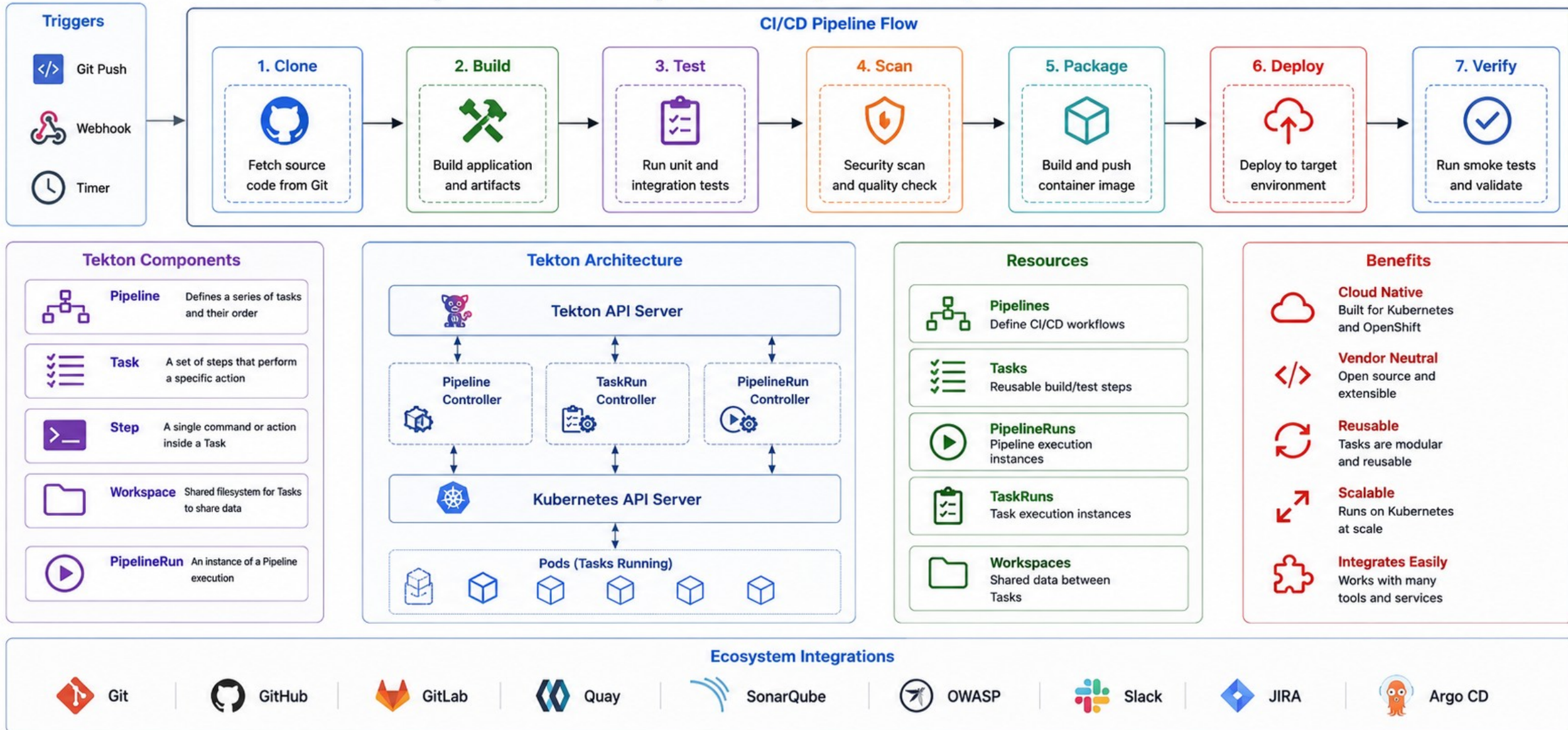


WHY PIPELINE AS CODE MATTERS

- Instead of manually configuring build steps in a UI, teams store the pipeline definition with the code
- Makes delivery repeatable, reviewable, and version-controlled

KEY TAKEAWAY Checkout, compile, unit test, integration test, static analysis, package, publish reports -- the typical CI pipeline shape. • Continuous delivery always leaves a person deciding when to release; continuous deployment ships automatically once checks pass. • Version-controlled pipeline definitions mean delivery logic gets reviewed, diffed, and rolled back exactly like application code.

OpenShift Pipelines (Tekton) Overview



SOURCE CONTROL STRATEGY · GIT BRANCHING MODELS

THREE BRANCHING MODELS

MODEL	BEST FOR
Trunk-Based	Fast-moving teams, CI/CD focused
GitFlow	Enterprise, regulated, scheduled releases
GitHub Flow	Simple, CI/CD, open source projects

STRATEGY ARCHITECTURE

- Central Repository -> Collaborate -> Pull Requests -> Quality Gates -> Merge & Release

STRATEGY PILLARS

- Security (least-privilege, branch protection), Collaboration, Traceability, Consistency, Automation

KEY TAKEAWAY Central repository, collaboration, pull requests, quality gates, and merge-and-release form the five-stage architecture behind reliable delivery. • Trunk-based suits fast-moving CI/CD teams; GitFlow suits regulated, scheduled releases; GitHub Flow suits simple, continuous deployment.

TRUNK-BASED DEVELOPMENT

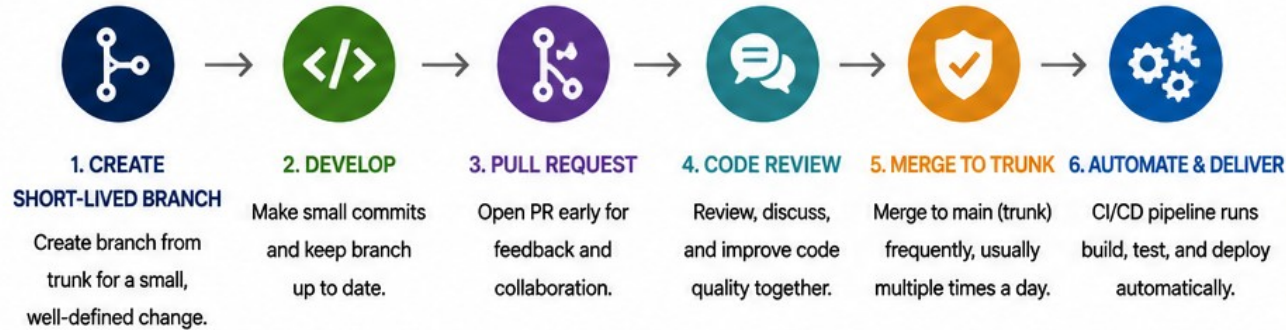
Trunk-Based Development is a branching strategy where developers work on short-lived feature branches and integrate changes into a single main branch (trunk) frequently. The trunk is always in a releasable state.



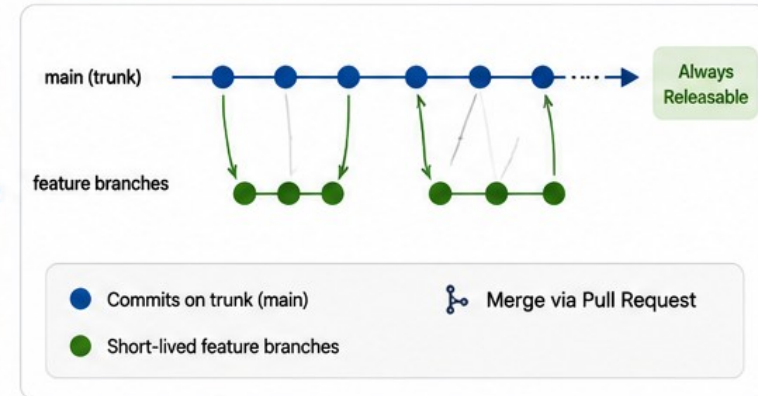
GOAL

Reduce integration pain, get feedback early, and enable continuous delivery.

TRUNK-BASED DEVELOPMENT WORKFLOW



TRUNK-BASED MODEL



TRUNK-BASED vs. LONG-LIVED BRANCHING

ASPECT	TRUNK-BASED DEVELOPMENT	LONG-LIVED BRANCHING
Branches	One main branch, short-lived feature branches	Multiple long-lived branches (develop, release, etc.)
Integration Frequency	Multiple times per day	Less frequent, at phase boundaries
Merge Complexity	Low (small changes)	High (large, complex merges)
Feedback	Fast feedback through CI and reviews	Delayed feedback
Risk	Lower risk, issues found early	Higher risk, issues found late
Release Readiness	Trunk is always releasable	Releases prepared on release branches
Best For	Agile teams, CI/CD, continuous delivery	Traditional teams, infrequent releases

BEST PRACTICES

- ✓ Keep feature branches short-lived (hours to a few days).
- ✓ Commit small, focused changes frequently.
- ✓ Pull latest changes from trunk before creating PR.
- ✓ Require code reviews and automated checks for all PRs.
- ✓ Use feature flags to toggle incomplete features.
- ✓ Automate build, test, and deployment on every commit.
- ✓ Fix broken builds immediately.
- ✓ Keep trunk stable, clean, and deployable.

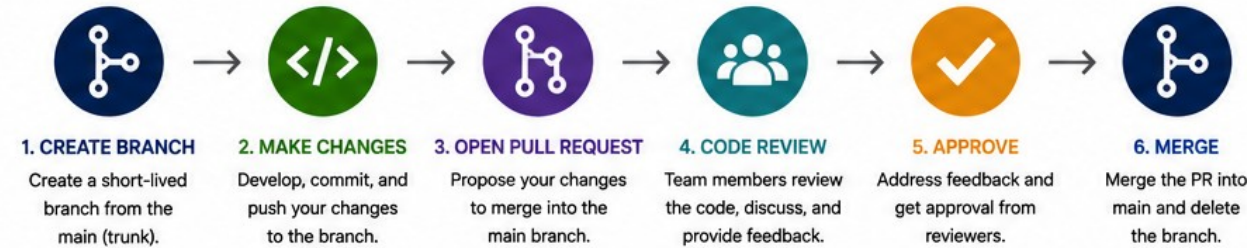


Small changes + automated testing + frequent integration
= faster delivery, higher quality, happier teams.

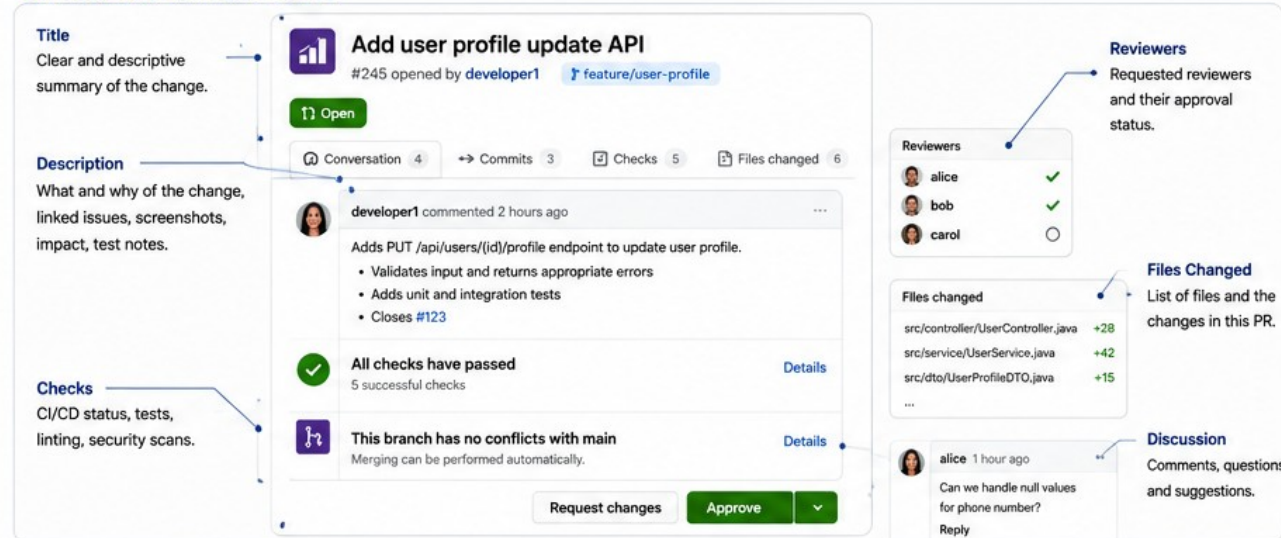
PULL REQUESTS AND CODE REVIEWS

Pull Requests (PRs) and Code Reviews ensure code quality, knowledge sharing, and alignment before changes are integrated into the main branch.

PULL REQUEST WORKFLOW



PULL REQUEST ANATOMY



GOAL

Deliver high-quality code together by catching issues early and sharing knowledge.

BENEFITS

- IMPROVED CODE QUALITY**
Bugs and issues are caught early through peer review.
- KNOWLEDGE SHARING**
Team members learn from each other's code and solutions.
- CONSISTENCY & STANDARDS**
Ensures code follows team standards and best practices.
- REDUCED RISK**
Review before merge reduces defects in production.
- AUDITABILITY & TRANSPARENCY**
Every change is visible, reviewed, and traceable.

BEST PRACTICES

- ✓ Keep branches small and focused.
- ✓ Write clear titles and detailed descriptions.
- ✓ Link related issues and tests.
- ✓ Request reviews early.
- ✓ Be respectful and constructive in feedback.
- ✓ Address feedback promptly.
- ✓ Automate checks and block on failures.
- ✓ Merge only when CI passes and reviewers approve.



GOOD REVIEWS FOCUS ON



CODE CORRECTNESS
Does it work as intended?



CODE QUALITY
Is it readable, simple, and maintainable?



SECURITY
Any security vulnerabilities or sensitive data?



TEST COVERAGE
Are tests adequate and meaningful?



DOCUMENTATION
Are docs and comments updated as needed?



PERFORMANCE
Any performance or scalability concerns?



Branch



Code



Commit



Pull Request



Review



Merge



Deploy



Monitor



GITHUB-BASED DEVELOPMENT WORKFLOW

A collaborative, branch-based workflow using GitHub to manage code, reviews, automation, and releases in a secure and transparent way.

WHY GITHUB?



Collaboration
at Scale



Security
Built-In



Automation
with Actions



Visibility &
Traceability



Integration
Ecosystem

WHAT IS IT?



GitHub provides a central platform for source code management, collaboration, CI/CD automation, issue tracking, and project visibility.

KEY BENEFITS



Improved collaboration and code quality



Faster and safer delivery



Auditability and compliance

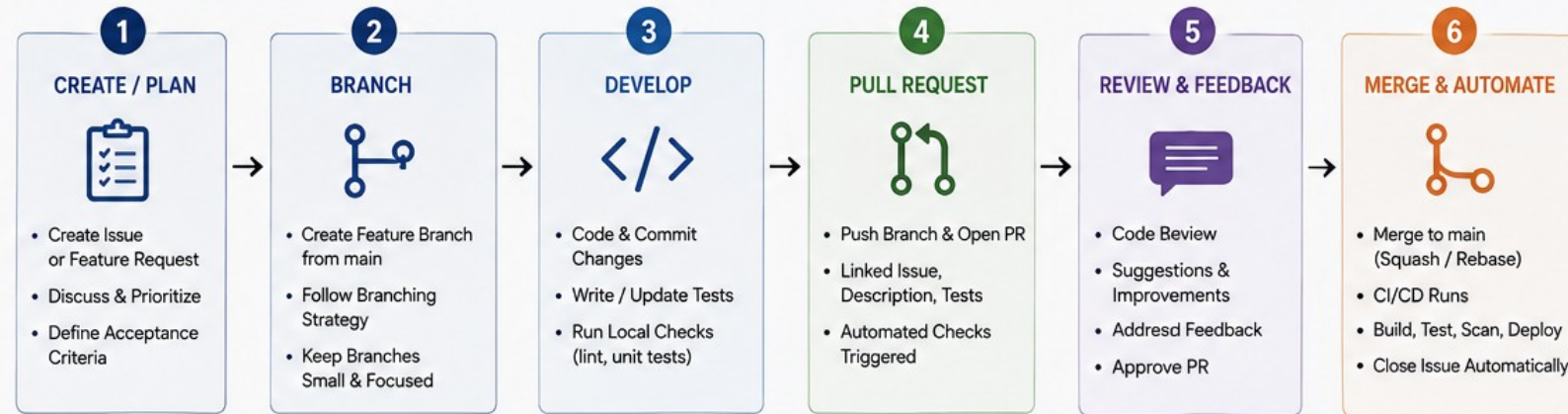


Automation reduces manual effort

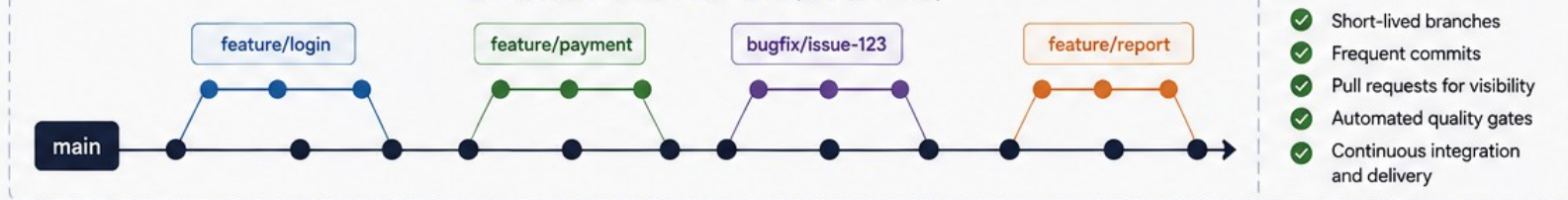


End-to-end visibility and traceability

TYPICAL GITHUB-BASED WORKFLOW



EXAMPLE BRANCHING STRATEGY (TRUNK-BASED)



AUTOMATION WITH GITHUB ACTIONS



INTEGRATIONS



BEST PRACTICES

- Use meaningful commit messages
- Follow naming conventions
- Keep PRs small and focused
- Write good tests
- Review before merge
- Automate quality gates

SEMANTIC VERSIONING · ARTIFACT VERSIONING AND TRACEABILITY · BUILD ONCE, DEPLOY MANY



THE VERSION FORMAT

- **MAJOR:** incompatible API changes
- **MINOR:** backward-compatible new functionality
- **PATCH:** backward-compatible bug fixes



EXAMPLES

- 1.5.0 adds functionality; 2.0.0 introduces breaking changes; 1.4.3 is a bug fix



ARTIFACT LIFECYCLE

- Code Commit -> Build -> Publish -> Deploy -> Trace
- Artifacts are immutable and versioned, stored in a secure artifact repository



WHAT TRACEABILITY ANSWERS

- What ran? Where? When? Built from what code? By which pipeline? With which config?



BUILD ONCE FLOW

- Code -> Build -> Package -> Verify -> Version & Store -> Deploy to dev, QA, staging, production



KEY PRINCIPLE

- Artifacts are immutable -- never change, only version. Promote, don't rebuild

KEY TAKEAWAY A major bump signals breaking changes, a minor bump signals new backward-compatible functionality, and a patch signals a safe bug fix. • Source code, pipeline run, artifact, container image, deployment, and runtime instance form one unbroken traceability chain. • Promoting one immutable, versioned artifact through dev, QA, staging, and production eliminates 'it works on my machine' problems entirely.

AUTOMATED QUALITY GATES · STATIC CODE ANALYSIS

FOUR QUALITY GATES

GATE	WHEN	PURPOSE
Code Quality Gate	After commit	Ensure code is clean, maintainable, secure
Build Quality Gate	After build	Ensure the build is successful and reliable
Security Quality Gate	After tests	Ensure the app is secure and compliant
Release Quality Gate	Before production	Ensure the release is stable and ready

WHAT STATIC ANALYSIS DETECTS

- Security vulnerabilities (SQLi, XSS, hardcoded secrets), bugs, code smells
- Compliance issues against OWASP Top 10, PCI-DSS, HIPAA, and custom rules

TOOLS

- SonarQube, Checkmarx, Veracode, Fortify SCA, GitHub CodeQL

KEY TAKEAWAY Code, build, security, and release quality gates each check a different concern at a different point in the pipeline. • Fixing a defect found in production costs roughly 30x what fixing it at the requirements stage would have cost.

MICROSERVICES PIPELINE STAGES

Contract testing and security/quality gates sit deliberately before packaging, so a broken API contract or vulnerability never becomes a shipped artifact.

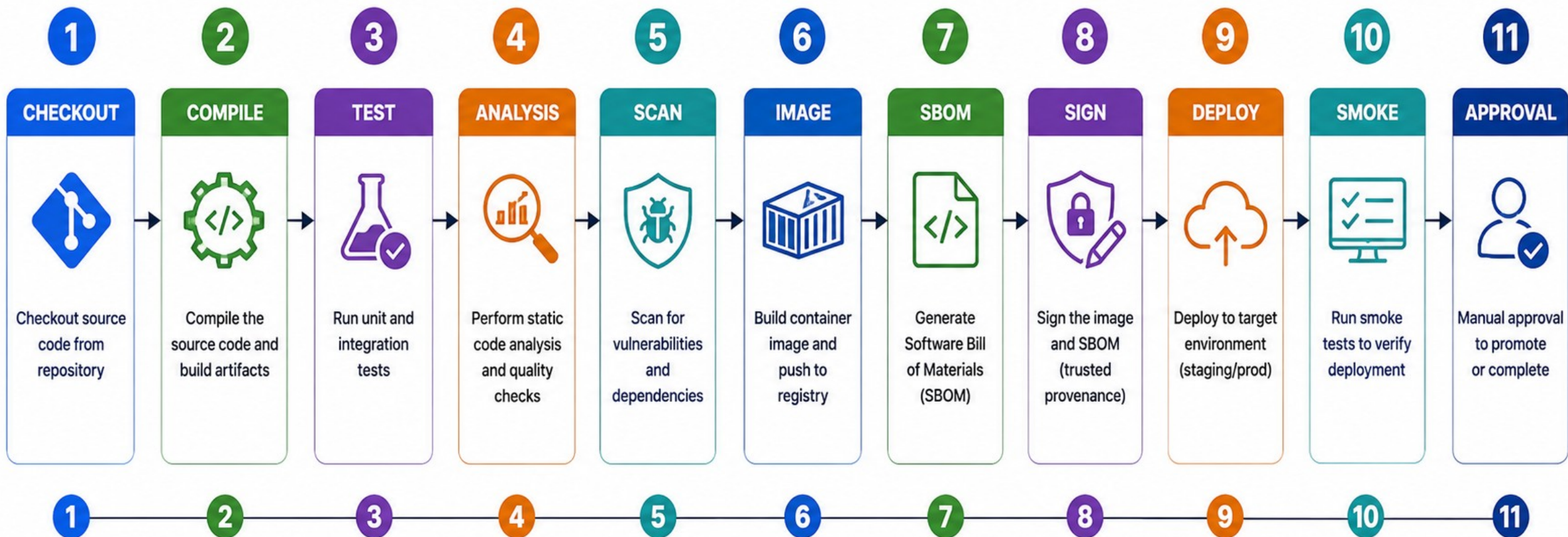


THE TEN PIPELINE STAGES

1. **Source checkout:** fetch the exact commit so the build is reproducible and traceable
2. **Dependency resolution:** resolve pinned versions from an approved repository
3. **Compile & build:** produce the artifact once, and promote that same artifact onwards
4. **Unit test:** verify business logic in isolation, fast enough to run on every commit
5. **Integration test:** exercise the service against real collaborators such as a database or broker
6. **Contract test:** prove the API still satisfies what consumers depend on before release
7. **Security & quality gates:** fail the build on vulnerabilities, coverage drops, or code smells
8. **Package artifact:** produce the versioned JAR that the rest of the pipeline promotes
9. **Build container image:** layer that artifact into a scanned, non-root image
10. **Push & deploy:** publish with an immutable tag, then roll out and smoke-test

KEY TAKEAWAY Contract testing and security/quality gates sit deliberately before packaging, so a broken API contract or vulnerability never becomes a shipped artifact.

Pipeline Stages



SOFTWARE COMPOSITION ANALYSIS (SCA)

Software Composition Analysis (SCA) identifies and manages risks in third-party components and open source dependencies that your application uses.



You can't secure what you can't see.

SCA gives you full visibility into the components in your software supply chain.

HOW SCA WORKS



1. IDENTIFY

Detect all open source and third-party components



2. ANALYZE

Match components against vulnerability and license databases



3. ASSESS

Evaluate risk based on severity, exploitability, and usage



4. REPORT

Provide actionable insights and prioritize remediation



5. REMEDIATE

Update, replace, or remove risky components

WHAT SCA DETECTS



VULNERABILITIES

Known CVEs in open source and third-party libraries



LICENSE RISKS

Incompatible, copyleft, or unapproved licenses



OUTDATED COMPONENTS

Components with available newer or secure versions



TRANSITIVE DEPENDENCIES

Risks coming from nested or indirect dependencies



POLICY VIOLATIONS

Components that violate organizational policies or allowlists/denylists

WHERE SCA FITS IN THE SDLC



Plan



Code



Build



Test



Deploy



Operate



SCA SCANS

- ✓ On every build and pull request
- ✓ Before release to any environment
- ✓ Continuously in production (monitor for new vulnerabilities)
- ✓ Integrated with CI/CD and artifact repositories



Goal: Build secure, compliant, and reliable software by managing the open source and third-party components in your applications.

KEY BENEFITS



REDUCE SECURITY RISK

Find and fix vulnerabilities before attackers do.



IMPROVE COMPLIANCE

Meet regulatory and license compliance requirements.



LOWER COST

Avoid costly breaches, legal issues, and emergency fixes.



IMPROVE SOFTWARE QUALITY

Use trusted, well-maintained components.



DRIVE TRANSPARENCY

Gain visibility into your software supply chain.

SCA TOOLS (EXAMPLES)

synopsys

Black Duck

Enterprise SCA and license compliance

sonatype

Nexus IQ

Identify & fix open source risk in real time

snky

Snyk

Vulnerability scanning for dependencies and containers

JFrog

JFrog Xray

Universal artifact analysis across binaries and packages

DEPENDABOT

Dependabot

Automated dependency updates and security alerts (GitHub)

WhiteSource

Open source risk management and policy enforcement

BEST PRACTICES

- ✓ Scan all dependencies (direct and transitive) on every build.
- ✓ Enforce policies and fail builds on high-risk vulnerabilities.
- ✓ Keep an allowlist of approved licenses and components.
- ✓ Prioritize fixes based on severity and exploitability.
- ✓ Continuously monitor for newly disclosed vulnerabilities.



Identify Risks



Prioritize Actions



Fix & Prevent



Stay Secure

SECRETS SCANNING

API keys, tokens, passwords, certificates, and cloud credentials are all things a secrets scanner is trained to spot in code and history.



HOW SECRETS SCANNING WORKS

- 1. Scan:** check commits, history, and build output for credential-shaped strings
- 2. Detect:** match against known patterns and entropy heuristics to find likely secrets
- 3. Alert:** notify the owning team immediately, because the exposure is already live
- 4. Remediate:** rotate the credential first, then remove it from history
- 5. Prevent:** add pre-commit hooks and pipeline gates so the next one never lands

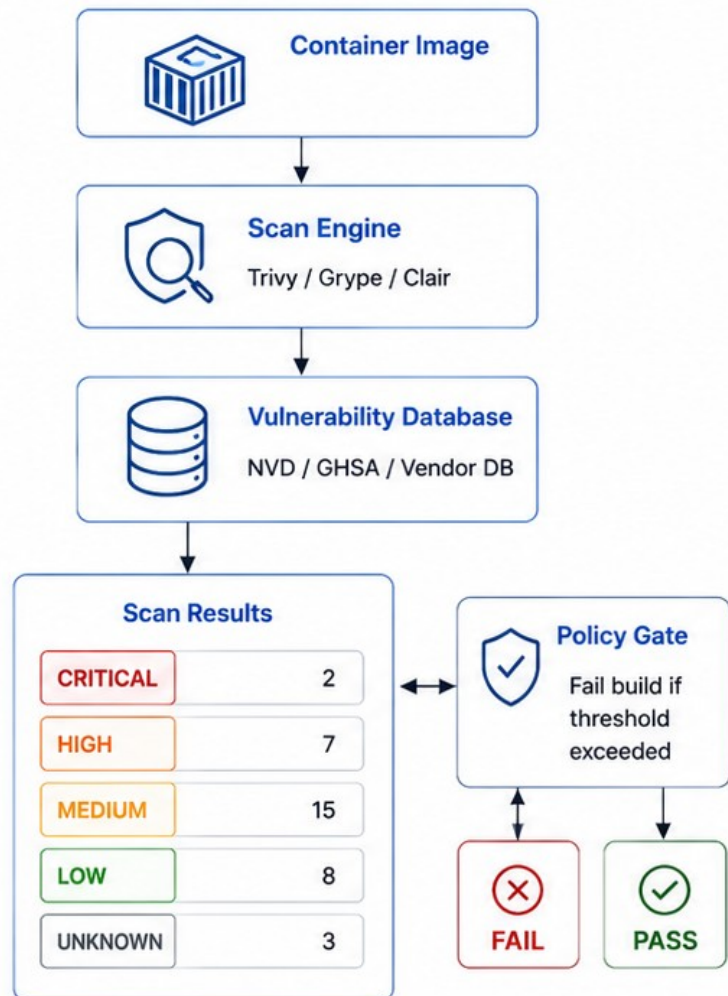


TOOLS

- GitHub Advanced Security, GitLab Secret Detection, Azure DevOps scanning, HashiCorp Vault

KEY TAKEAWAY API keys, tokens, passwords, certificates, and cloud credentials are all things a secrets scanner is trained to spot in code and history.

VULNERABILITY SCANNING



SBOM (SOFTWARE BILL OF MATERIALS)

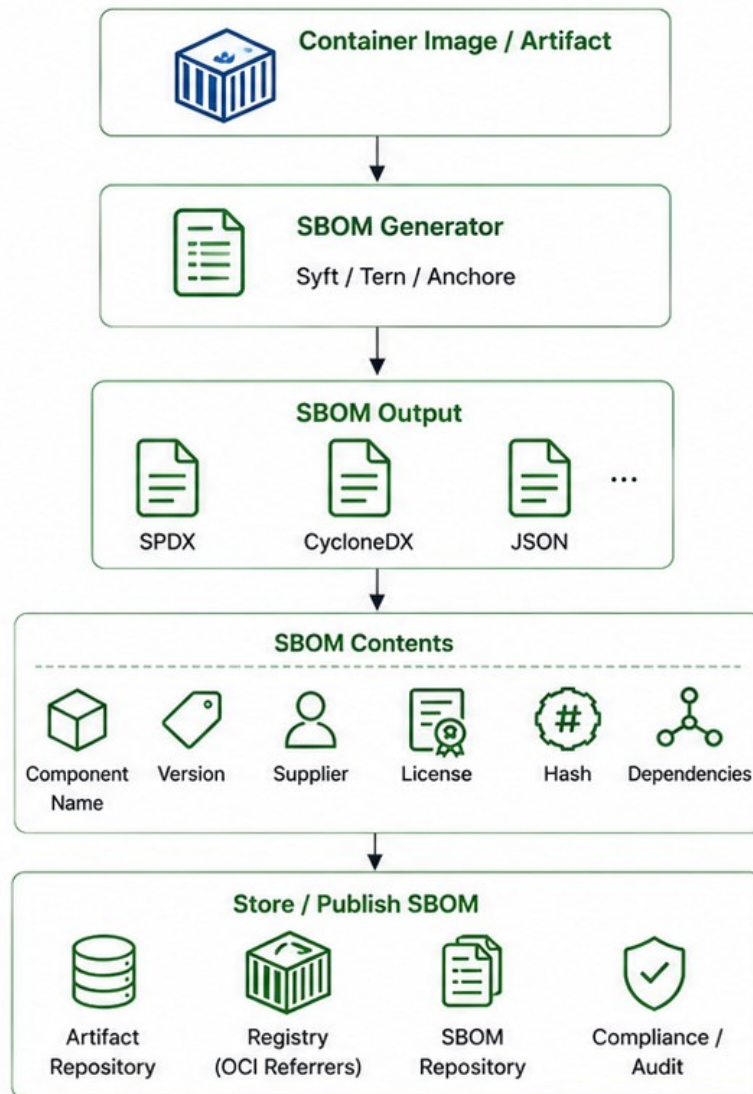
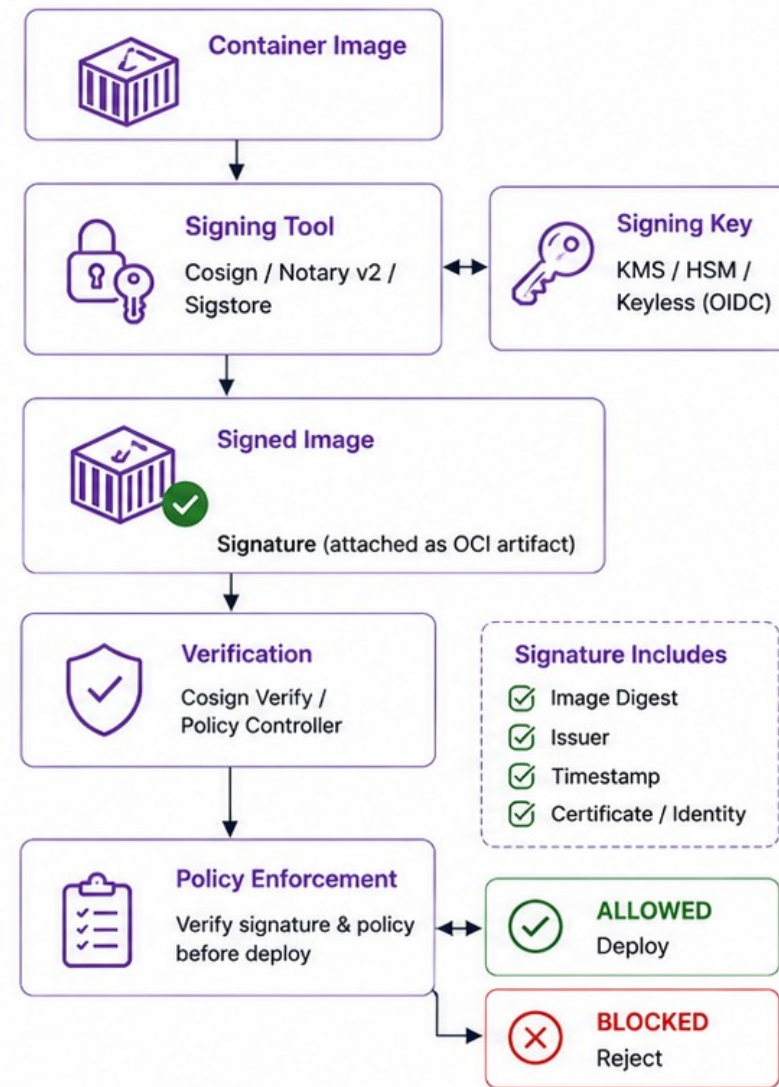


IMAGE SIGNING



CONTAINER IMAGE SCANNING · SHIFT-LEFT TESTING AND SECURITY · DEVSECOPS FOR MICROSERVICES



WHAT IT DETECTS

- Known vulnerabilities (CVEs), risky base images, misconfigurations, malware, secrets baked into layers



TOOLS

- Trivy (Aqua), Anchore Gype, Snyk Container, Red Hat Quay, Amazon ECR scanning



INTEGRATING SECURITY AND TESTING EARLY

- **Security:** threat modeling -> secure design review -> SAST/IDE scanning -> DAST/IAST -> runtime protection
- **Testing:** requirements validation -> test planning -> unit -> integration -> UAT/E2E -> monitoring



COST OF FIXING DEFECTS

- 1x at requirements, 5x at design, 10x at coding, 15x at testing, 30x in production



THE SEVEN-STAGE DEVSECOPS LIFECYCLE

- Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate, with continuous feedback looping back



SECURITY ACROSS THE STACK

- Application, container, platform, data, identity, and governance layers all need their own controls

KEY TAKEAWAY Scanning after every build and failing the pipeline on critical findings keeps vulnerable images from ever reaching the registry.

- Shift-left moves testing and security earlier into plan, design, and code, instead of waiting until test or deploy to find problems.
- Threat modeling at plan, dependency scanning at build, and runtime protection at operate turn security into a continuous thread, not a gate.

SOURCE CHECKOUT AND DEPENDENCY RESOLUTION · COMPILATION AND UNIT TESTING · INTEGRATION AND CONTRACT TESTING



WHAT HAPPENS IN THIS STAGE

- Clone/pull latest code, validate branch and commit, read build files (pom.xml/build.gradle)
- Resolve and download dependencies, using caching and mirrors for speed



WHAT HAPPENS IN THIS STAGE

- Source code checkout -> compile (download dependencies) -> run unit tests -> quality gate



BEST PRACTICES

- Keep unit tests fast, isolated, and deterministic; follow Arrange-Act-Assert
- Mock external dependencies; fail the build on any compilation error or test failure



TWO COMPLEMENTARY TEST TYPES

- **Integration testing:** ensures multiple services or components work together correctly
- **Contract testing:** verifies that producers and consumers agree on the API contract



PACT CONTRACT FLOW

- Consumer generates a contract -> provider verifies against that same contract

KEY TAKEAWAY Pinned dependency versions and a local cache turn a potentially flaky download step into a fast, reproducible one. • The Arrange-Act-Assert pattern in a JUnit 5 test keeps each unit test fast, isolated, and easy to read at a glance. • A Pact contract generated by the consumer and verified by the provider catches a breaking API change before it ever reaches production.



APPLICATION PACKAGING

Package the application and its dependencies into a distributable artifact ready for deployment to any environment.

Module 18
CI/CD with GitHub
and Jenkins

Day 4



Goal: Create a versioned, portable, and immutable package that contains everything needed to run the application consistently across environments.

APPLICATION PACKAGING PROCESS



1. Source Code

Compiled and tested source code is ready.



2. Dependency Resolution

Dependencies are resolved and downloaded.



3. Build

Application is compiled and resources are processed.



4. Package

Artifacts are created (JAR/WAR or container image) with version.



5. Store

Artifacts are stored in an artifact repository for traceability.

TYPES OF PACKAGING ARTIFACTS

JAR / WAR

Traditional Java artifacts for Spring Boot applications.



- Executable JAR with embedded server (Spring Boot).
- WAR for deployment to external application servers.



CONTAINER IMAGE

Containerized application with all runtime dependencies.



- Includes app, runtime, libraries, and config.
- Portable and consistent across environments.
- Ideal for cloud-native deployments.

WHAT'S INCLUDED?



- ✓ Compiled application classes
- ✓ Dependencies (libraries)
- ✓ Configuration files
- ✓ Static resources
- ✓ Application metadata
- ✓ Bootstrap and runtime components

EXAMPLE: SPRING BOOT JAR CONTENTS



- BOOT-INF/
 - classes/
 - lib/
- META-INF/
 - MANIFEST.MF

- ✓ BOOT-INF/classes: Your compiled application.
- ✓ BOOT-INF/lib: All dependencies.
- ✓ META-INF/MANIFEST.MF: Application entry point.

WHY IT MATTERS



Ensures consistency across all environments.



Enables immutable and repeatable deployments.



Speeds up deployments and reduces manual errors.



Improves traceability and rollbacks with versioned artifacts.

BEST PRACTICES



- ✓ Use semantic versioning for all artifacts.
- ✓ Keep build outputs clean and reproducible.
- ✓ Externalize configuration; avoid environment-specific hardcoding.
- ✓ Store artifacts in a secure, centralized repository.
- ✓ Scan packages for vulnerabilities before publishing.
- ✓ Build once, deploy many.



CONTAINER IMAGE CREATION

Build a lightweight, secure, and reproducible container image that packages the application and everything it needs to run.

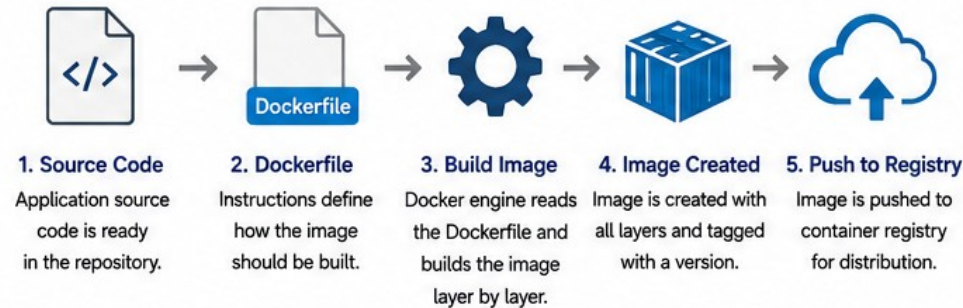
Module 18
CI/CD with GitHub
and Jenkins

Day 4



Goal: Create a versioned container image that includes the application and its runtime dependencies, ready to run consistently in any environment.

CONTAINER IMAGE CREATION PROCESS



EXAMPLE: DOCKERFILE FOR SPRING BOOT APPLICATION

```
1 FROM eclipse-temurin:17-jre-alpine
2 WORKDIR /app
3 COPY target/myapp.jar app.jar
4 EXPOSE 8080
5
6 ENV JAVA_OPTS="-XX:+UseG1GC -XX:MaxRAMPercentage=75.0"
7
8 ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
9
10
```

RESULTING IMAGE

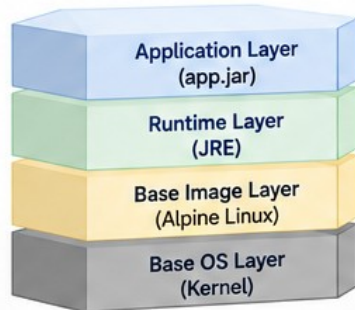


myapp:1.0.0

(Versioned Image)

Ready to run anywhere
Docker is available.

IMAGE LAYERS (EXAMPLE)



Images are built in layers. Only changed layers are rebuilt, making builds faster and efficient.



Smaller size.
Faster builds.
Better caching.

BEST PRACTICES

- ✓ Use minimal base images (e.g., Alpine).
- ✓ Run as non-root user.
- ✓ Use multi-stage builds to reduce image size.
- ✓ Leverage .dockerignore to exclude unnecessary files.
- ✓ Tag images with meaningful versions (e.g., 1.0.0, latest).
- ✓ Scan images for vulnerabilities before publishing.
- ✓ Keep images immutable. Build once, run anywhere.

WHAT'S INCLUDED IN THE IMAGE?



Application artifact (e.g., JAR/WAR)



Runtime (JRE/JDK)



Configuration files



Environment variables



Dependencies and libraries



Startup command and metadata



Why it matters: A well-built container image ensures consistency, portability, security, and speed across all environments.



**Build once.
Deploy everywhere.**

PUBLISHING TO AN ARTIFACT REGISTRY · DEPLOYING ACROSS ENVIRONMENTS · ENVIRONMENT PROMOTION STRATEGIES



WHY REGISTRIES MATTER

- Build image -> scan image -> push image to registry -> platform pulls image -> run container
- Registries provide authentication, authorization, scanning, audit logging, and retention policies



DEPLOYMENT ACROSS ENVIRONMENTS

- Retrieve the built artifact once, then deploy the same version to each environment in sequence
- Configuration (URLs, credentials, replica counts) changes per environment; the artifact never does



PROMOTION FLOW

- Dev (auto-deploy on merge) -> QA/Test (automated testing) -> Staging (pre-prod validation) -> Production (manual/auto approval)
- The same artifact or image is promoted across every stage -- never rebuilt

KEY TAKEAWAY Immutable, versioned tags pushed to a private registry make every deployed artifact traceable back to its exact build. • Environment-specific ConfigMaps and Secrets let one immutable image behave correctly in dev, staging, and production alike. • Dev auto-deploys on merge, staging requires validation, and production requires manual or policy-gated approval.

PIPELINE VARIABLES AND CREDENTIALS · PIPELINE APPROVALS AND GOVERNANCE



TYPES OF PIPELINE SECRETS

- GitHub tokens, registry credentials, deployment tokens, database passwords, signing keys, API keys



BEST PRACTICES

- Store secrets in secure credential stores, never in source code or pipeline logs
- Apply least privilege and rotate credentials periodically

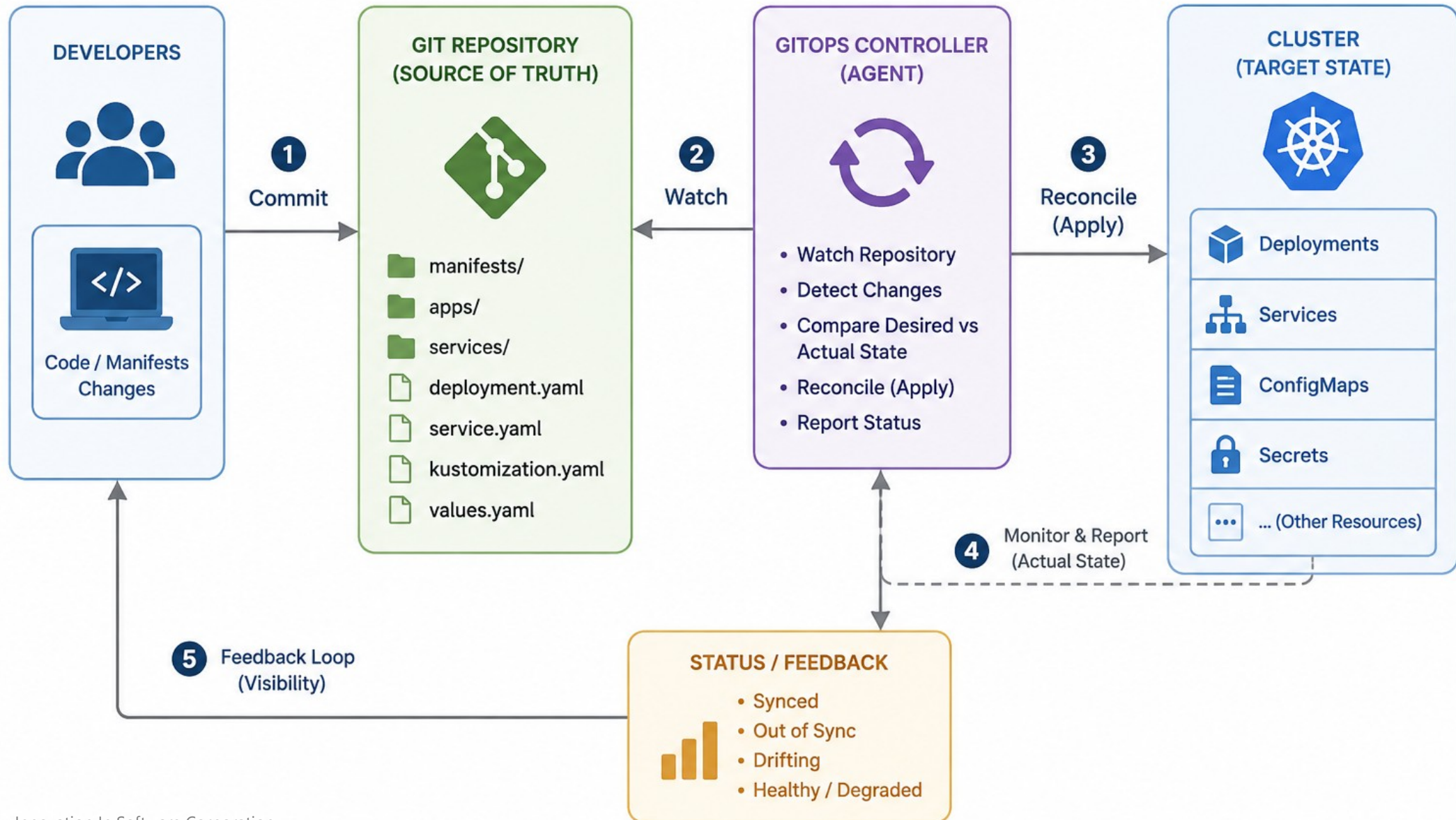


WHY APPROVAL GATES EXIST

- Production releases often require a human decision point even in a fully automated pipeline
- Governance policies define who can approve, and what evidence (tests, scans) must be attached

KEY TAKEAWAY Registry tokens, deployment credentials, and database passwords for test environments all need to be injected securely, not committed. • Compliance-driven governance requires that not every pipeline stage should run unattended -- production deploys are the deliberate exception.

GitOps Overview (Conceptual)



GITHUB ACTIONS CONCEPTS · JENKINS ARCHITECTURE AND COMPONENTS



KEY CONCEPTS

- **Workflow:** an automated YAML-defined process that runs on events
- Event, Job, Step, Action, Runner -- the building blocks of every workflow



JENKINS MASTER (CONTROLLER)

- Web UI/CLI/REST API, core services (scheduling, security, plugin management), data store



EVENT TRIGGERS

- push, pull_request, schedule, workflow_dispatch, release, issues

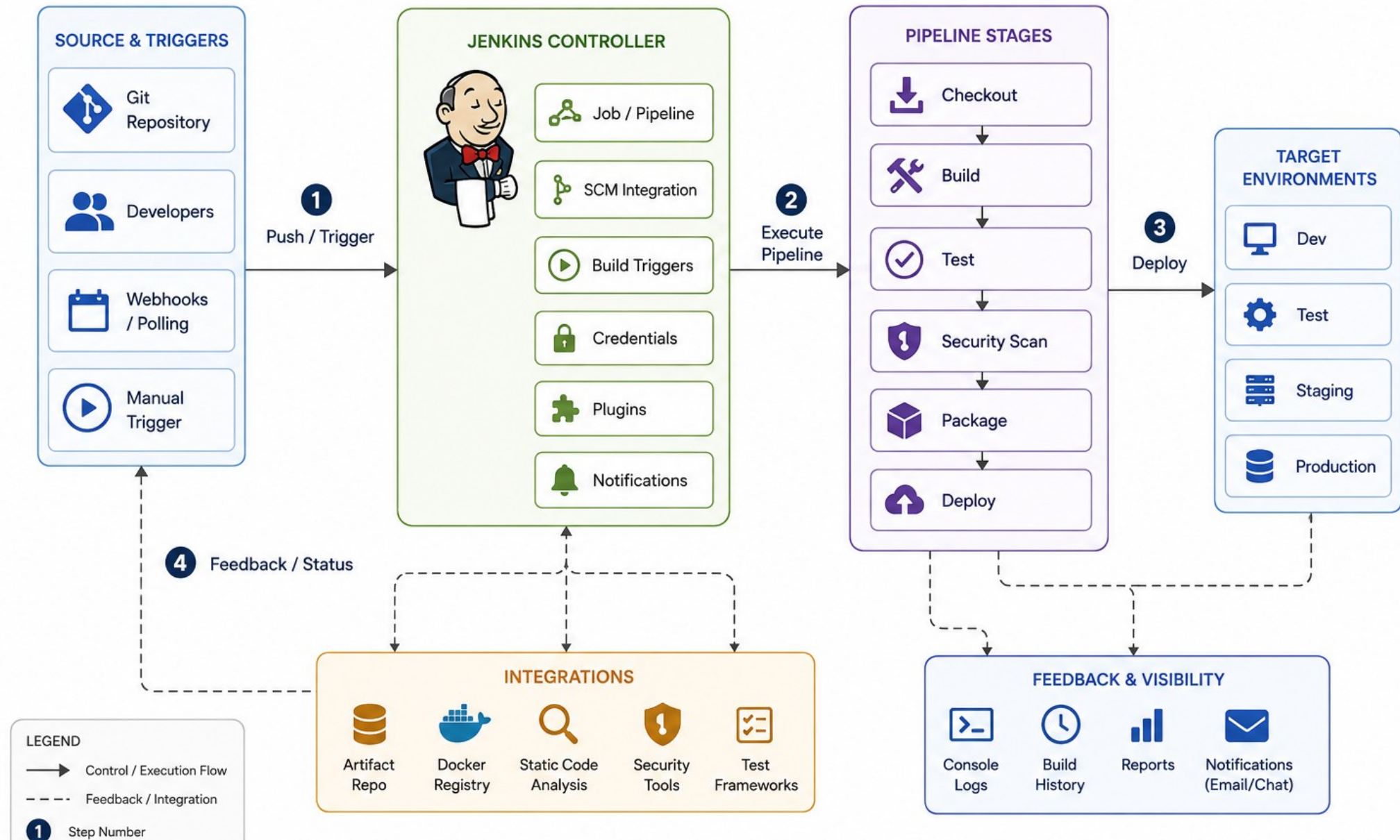


AGENTS (BUILD NODES)

- Distributed nodes that execute builds, run tests, package artifacts, and deploy applications

KEY TAKEAWAY A push event triggers a workflow, which runs jobs in parallel, each executing its steps sequentially on a runner. • Jobs, plugins, builds, workspaces, and credentials are the core components that let a Jenkins controller coordinate work across many agents.

Jenkins as Alternate Enterprise Pattern (awareness)



DECLARATIVE JENKINS PIPELINES

Declarative Pipeline is a higher-level abstraction for defining CI/CD pipelines in Jenkins using a simple, structured, and readable syntax. It provides conventions, reduces boilerplate code, and improves consistency.

WHY DECLARATIVE PIPELINES?



Simple and Readable



Opinionated Best Practices



Safer and Reliable



Visualization & Blue Ocean



Easier Maintenance

KEY FEATURES

- Structured syntax with predefined sections and directives
- Built-in error handling and post conditions
- Easy parallel execution and reusability
- Extensible with shared libraries and custom steps
- Great integration with Jenkins ecosystem

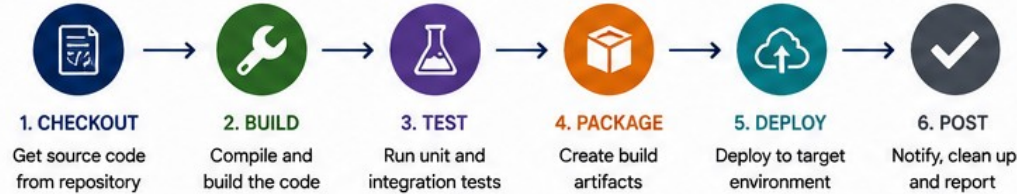
BENEFITS

- ✓ Faster pipeline creation
- ✓ Consistency across teams
- ✓ Reduced errors and maintenance
- ✓ Better visualization (Blue Ocean)
- ✓ Supports advanced pipeline features

COMMON DIRECTIVES

agent	Where the pipeline runs
environment	Define environment variables
options	Set pipeline-wide options
stages	Group of sequential or parallel stages
steps	Tasks executed in a stage
post	Actions after pipeline completion

TYPICAL PIPELINE FLOW



PIPELINE STRUCTURE

agent	Defines where the pipeline runs (any, label, docker, etc.)
environment	Sets environment variables for the pipeline
options	Configures pipeline options (timeout, retry, timestamps, etc.)
stages	Defines the sequence of stages in the pipeline
steps	Contains the steps or tasks executed in a stage
post	Defines actions to run after the pipeline (success, failure, always)

EXAMPLE STAGE WITH STEPS

```
stage('Build') {
  steps {
    echo 'Building the application...'
    sh 'mvn clean package'
    archiveArtifacts artifacts: 'target/*.jar'
  }
}
```

- echo - Print messages to the console
- sh - Execute shell commands
- archiveArtifacts - Save build artifacts

PIPELINE VISUALIZATION (BLUE OCEAN)



SYNTAX EXAMPLE

```
pipeline {
  agent any

  environment {
    APP_NAME = 'my-app'
    MAVEN_OPTS = '-Xmx1024m'
  }

  options {
    timestamps()
    buildDiscarder(logRotator(numToKeepStr: '10'))
    disableConcurrentBuilds()
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }
    stage('Build') {
      steps {
        sh 'mvn clean package -DskipTests'
      }
    }
    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
  }

  post {
    success {
      echo 'Build Successful!'
    }
    failure {
      echo 'Build Failed!'
    }
    always {
      cleanWs()
    }
  }
}
```

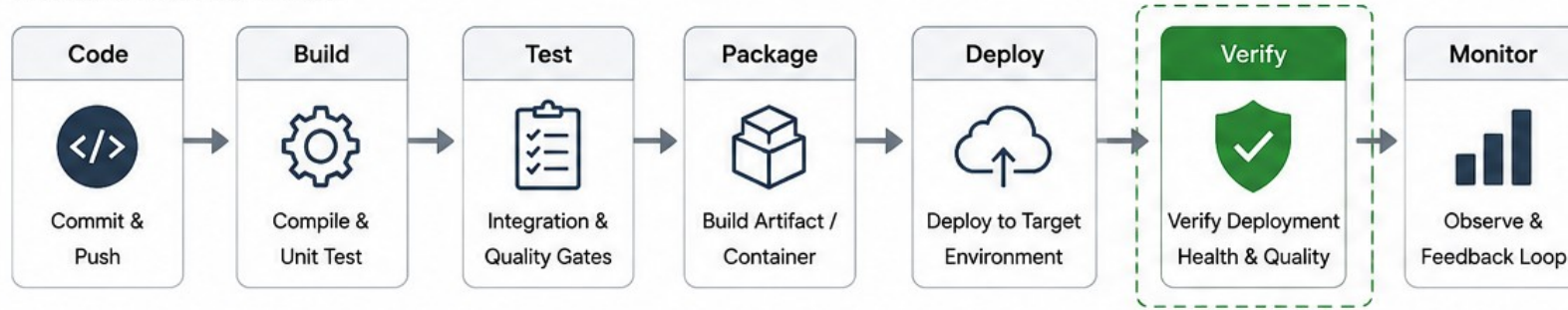
BEST PRACTICES

- ✓ Keep pipelines simple and modular
- ✓ Use meaningful stage and step names
- ✓ Leverage shared libraries
- ✓ Handle errors gracefully with post
- ✓ Use environment variables for configuration
- ✓ Avoid hardcoding credentials
- ✓ Archive artifacts and logs

DEPLOYMENT VERIFICATION

Confirm that what we deployed is running correctly, is healthy, and meets quality requirements.

Where Verification Fits



What We Verify

Deployment Success



- Deployment completed without errors
- Correct version and configuration applied

Availability



- Service endpoints are reachable
- No 5xx errors
- Healthy status codes

Health Checks



- Liveness probes pass
- Readiness probes pass
- Dependencies are healthy

Functional Tests



- Smoke tests pass
- Key user journeys validated
- API contract tests pass

Performance



- Response time within SLOs
- Error rate within thresholds
- Resource usage normal

Security & Compliance



- TLS certificates valid
- Security scans pass
- Compliance policies verified

Verification Pillars



Health

Ensure the application and infrastructure are healthy and responsive.



Functionality

Validate critical business flows and API contracts work as expected.



Performance

Confirm response times and throughput meet expectations.



Security

Verify security controls, certificates, and access policies are enforced.



Compliance

Ensure compliance checks, policies, and audit requirements pass.

Verification Techniques



Automated Smoke Tests



API Health Checks



Synthetic Monitoring



Security Scanning



Policy as Code Validation



Canary / Blue-Green Validation

Best Practices

- ✓ Automate verification in the pipeline
- ✓ Verify in production-like environment
- ✓ Fail fast and stop promotion on failure
- ✓ Generate actionable reports and alerts
- ✓ Include rollback decision points

Outcome



High confidence that the release is safe, functional, and ready for users.



AUTOMATED ROLLBACK STRATEGIES · PIPELINE NOTIFICATIONS AND AUDITABILITY · CI-CD PIPELINE SECURITY



ROLLBACK OPTIONS

- Redeploy the previous version, shift traffic back to the previous environment
- Disable a feature flag, or roll forward with an emergency fix



NOTIFICATIONS AND AUDIT TRAILS

- Notify teams of pipeline success, failure, and approval requests (Slack, email)
- Maintain audit logs of every build, deploy, and approval decision for compliance



SECURING THE PIPELINE

- Least-privilege service accounts, short-lived credentials where supported
- Secrets scanning, dependency scanning, and image scanning built into every run
- Restrict who can modify pipeline definitions and approve production deployments

KEY TAKEAWAY Redeploying the previous version, switching traffic back, or disabling a feature flag are the three fastest paths back to stability.

- Notifications keep teams informed in real time; audit logs let anyone reconstruct who deployed what, when, and why, after the fact.
- A compromised pipeline credential can reach source code, registries, and production deployments all at once -- it deserves top-tier protection.

TROUBLESHOOTING FAILED PIPELINES

Working backward from the first failed stage, checking its logs and inputs, resolves most pipeline failures in minutes, not hours.



TROUBLESHOOTING APPROACH

- Identify the first failed stage and read its full console/log output
- Check credentials, dependency availability, and environment configuration for that stage
- Reproduce locally where possible before re-running the full pipeline

KEY TAKEAWAY Working backward from the first failed stage, checking its logs and inputs, resolves most pipeline failures in minutes, not hours.

EXERCISE 4.3 — TRACE A PIPELINE STAGE OWNERSHIP MAP

Module 11 checkpoint C — Who owns scan / SBOM / sign / promote.



OBJECTIVE

- Who owns scan / SBOM / sign / promote
- 15 min · worksheet checkpoint
- **Worksheet:**
labs/day-04/lab4/starter/pipeline/OWNERSHIP.md



DO THIS

1. Open labs/day-04/lab4/starter/pipeline/OWNERSHIP.md and fill scan / SBOM / sign / promote
2. CRITICAL CVE fails the gate. SBOM is the ingredients list. Unsigned images must not run
3. Participants execute and verify — they do not manage signing keys
4. GitOps is conceptual. Do not install Jenkins or GitHub Actions runners



EXPECTED RESULT

- A completed ownership map. Compare with labs/day-04/lab4/solution/pipeline/OWNERSHIP.md when asked.
- Fill OWNERSHIP.md in Lab 4 before run-pipeline-locally.ps1. Jenkins is awareness only.

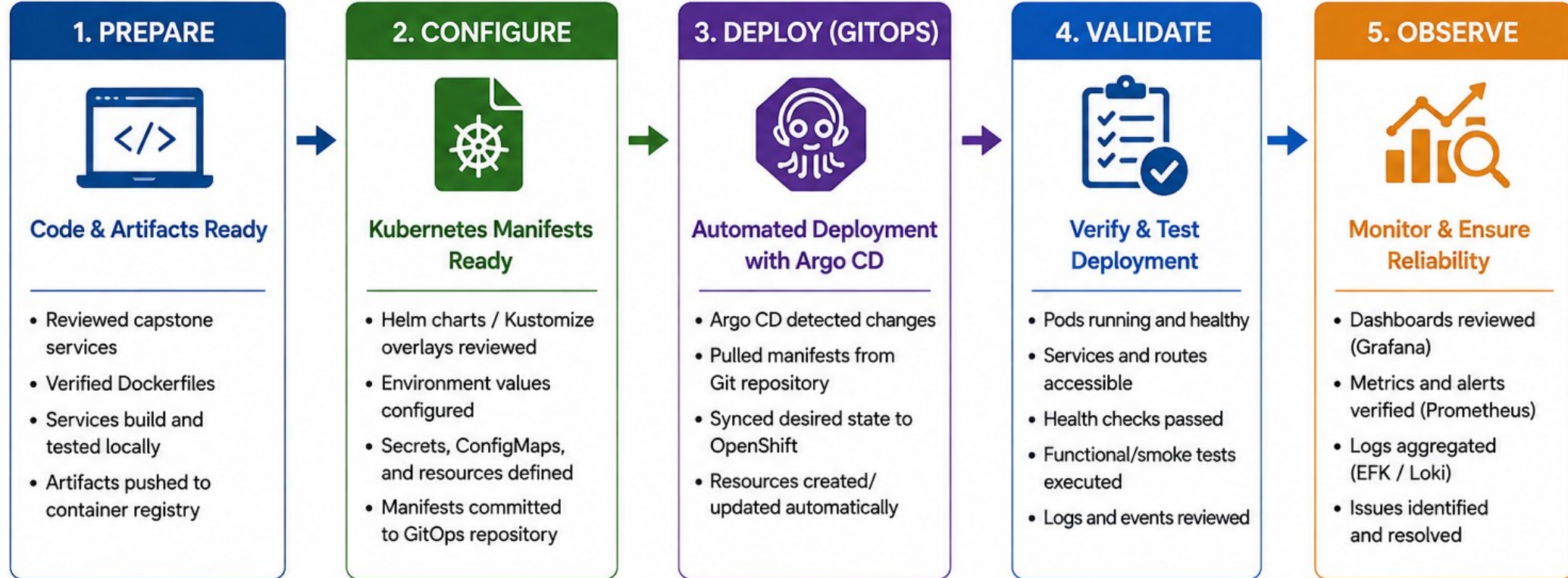
KEY TAKEAWAY A completed ownership map. Compare with labs/day-04/lab4/solution/pipeline/OWNERSHIP.md when asked.

Day 4 Summary



KEY TAKEAWAY

We built a repeatable, automated path to deploy the capstone services to OpenShift using GitOps, validated the deployment, and ensured observability and reliability.



OUTCOMES



Capstone services deployed on OpenShift



GitOps pipeline validated end-to-end



Environment is healthy and operational



Observability and alerts confirmed



Platform ready for continued development

Knowledge Check

1 What is GitOps and how does it work?



- Git is the single source of truth
- Desired state defined in Git
- Automation reconciles actual state

2 What role does Argo CD play in GitOps?



- Watches Git repository
- Detects changes
- Syncs desired state to the cluster

3 What is the flow from commit to running services?



- Commit manifests to Git
- Argo CD detects changes
- Syncs to OpenShift
- Resources updated and running

4 What are the key benefits of GitOps?



- Declarative and versioned
- Automated and consistent
- Auditable and traceable
- Safer and more reliable deployments

5 What are Kubernetes manifests used for?



- Define desired state
- Describe resources (Deployments, Services, Routes, etc.)
- Stored in Git

6 What is a Helm chart and why is it used?



- Package of Kubernetes manifests
- Reusable and parameterized
- Simplifies deployment and management

7 Where are container images stored?



- In a container registry (e.g., Quay)
- Referenced by image name and tag in manifests

8 What is the purpose of ConfigMaps and Secrets?



- ConfigMaps store non-sensitive configuration
- Secrets store sensitive data (passwords, keys, tokens)
- Injected into applications

9 How do you verify a successful deployment?



- Check pod status (Running)
- Verify services and routes
- Review logs
- Monitor metrics and alerts

10 What tools help you monitor and observe the platform?



- Prometheus for metrics
- Grafana for dashboards
- Alertmanager for alerts
- EFK / Loki for logs

KEY TAKEAWAYS



Git is the source of truth for your infrastructure.



Argo CD automates deployments by reconciling desired and actual state.



Helm and manifests make deployments reusable and consistent.



ConfigMaps and Secrets keep configuration secure and portable.



Observability ensures reliability and helps resolve issues quickly.

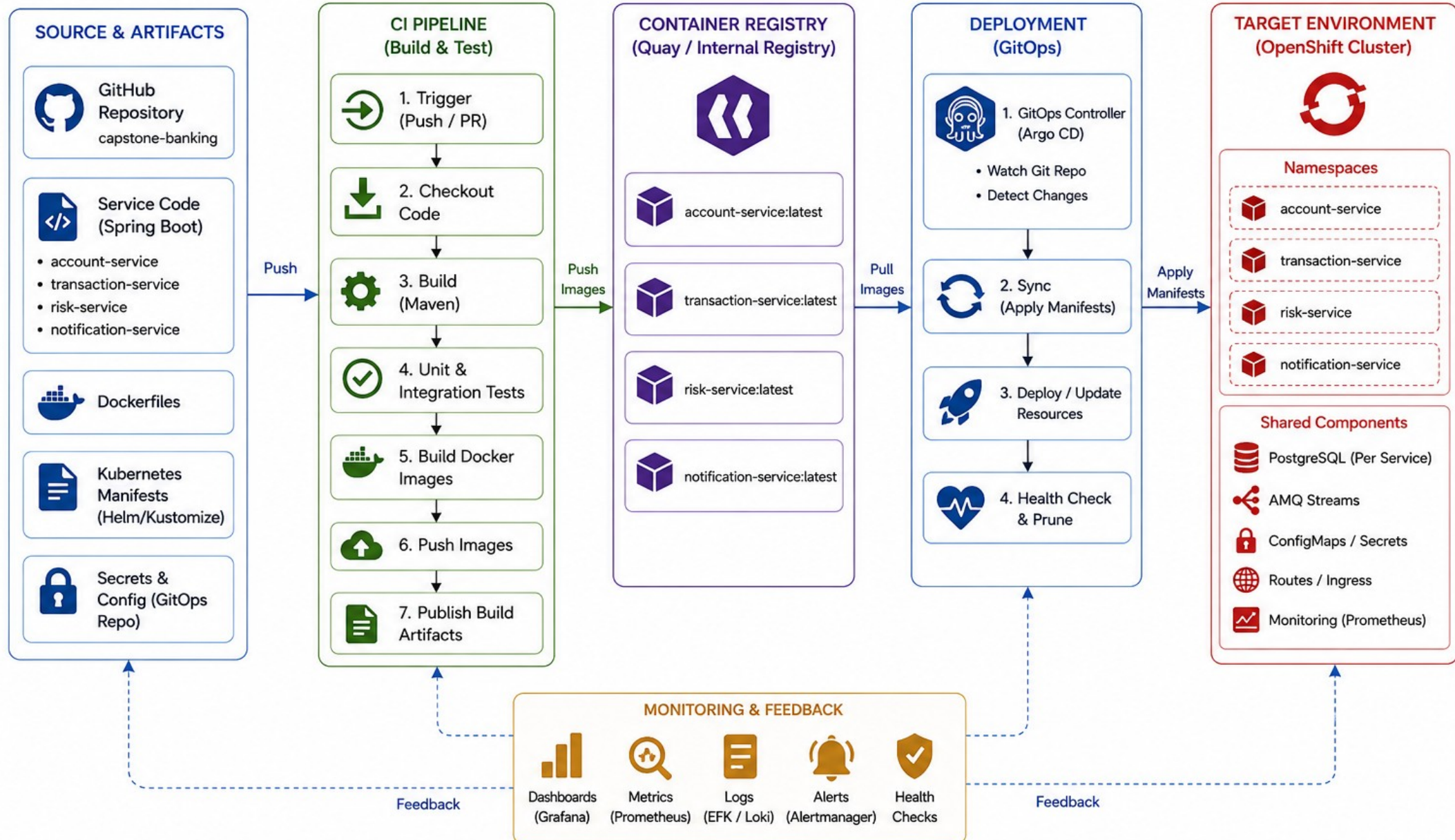
LAB 4

Lab 4 — Deploy the Capstone Services

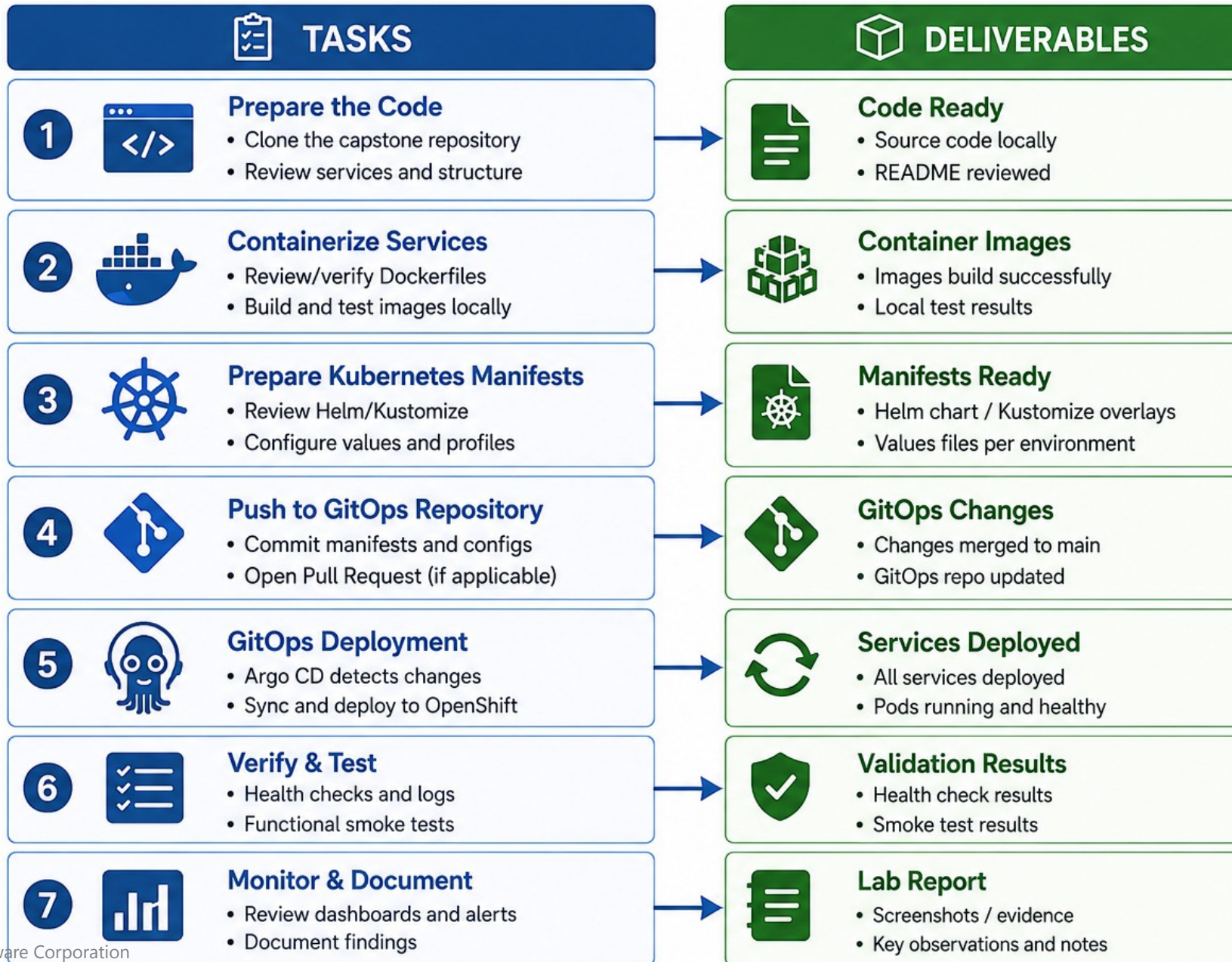
Services observable, containerized, and deployed via a completed CI/CD pipeline.



Lab 4 Overview - Deploy the Capstone Services



Lab Tasks and Deliverables



LAB 4 — DEPLOY THE CAPSTONE SERVICES — OVERVIEW

Day 4 lab: what you build and how it grows the capstone.



WHAT YOU WILL FINISH WITH

- Multi-stage Containerfiles that run as user md287, not root
- Account on 8081 and Transaction on 8082 from Docker images (Compose, then OpenShift)
- Liveness and readiness probes hitting Actuator
- ConfigMaps for URLs, Secrets for passwords and the classroom JWT applied on OpenShift
- Resource requests and limits on the OpenShift Deployments
- Working Routes on the classroom cluster
- Logs that still print correlationId= and never print Authorization
- **A completed prepared pipeline:** scan gate, SBOM, signature verify
- A version bump (1.0.0 → 1.0.1) and oc rollout undo back to 1.0.0

KEY TAKEAWAY Lab 4 builds Deploy the Capstone Services -- the working code behind today's concepts, and the next layer of the banking capstone.

LAB 4 — DEPLOY THE CAPSTONE SERVICES — BEFORE YOU START

Check these before the lab clock starts.

 KNOWLEDGE YOU NEED (FROM DAY 4)

Day 4 idea	How it appears in this lab
Liveness vs readiness vs startup	HTTP GET /actuator/health/liveness and /actuator/health/readiness. Startup probe gives Spring time to boot.
Correlation ID	Header X-Correlation-Id still lands in logs as correlationId=. That is the classroom trace handle.
Metrics	Compose exposes health,info,metrics. Call /actuator/metrics with a JWT. Health stays public.
Non-root image	USER md287 in the Containerfile. OpenShift also sets runAsNonRoot: true.
Config vs secret	JDBC URL in a ConfigMap. DB password and JWT secret in a Secret.
Immutable tags	Promote 1.0.0 / 1.0.1, not :latest.
Pipeline gates	CRITICAL CVE → fail. SBOM is the ingredients list. Unsigned images must not run.

KEY TAKEAWAY Settle prerequisites and environment first -- most lab time is lost to setup, not to code.

LAB 4 — DEPLOY THE CAPSTONE SERVICES — LAB STEPS

7 steps, worked in order.

LAB STEPS

- Step 1 — Confirm the Lab 3 images you will ship
- Step 2 — Finish the Containerfiles (non-root)
- Step 3 — Exercise 4.2 recap (layered image)
- Step 4 — Run the stack and prove probes, metrics, and correlation

LAB STEPS (CONTINUED)

- Step 5 — Log in, apply manifests, and push images (OpenShift — required)
- **Step 6 — Prepared pipeline:** SBOM, signature, scan gate
- Step 7 — Deploy a new version and roll it back (OpenShift — required)

KEY TAKEAWAY Work the steps in order -- each one builds on the last.

LAB 4 — DEPLOY THE CAPSTONE SERVICES — SUCCESS CRITERIA

You are done when every box below is ticked.

SUCCESS CRITERIA

- Account and Transaction images build from Lab 3 solution context
- Container process is not root (docker run --rm --entrypoint id ...)
- **Compose stack:** liveness, readiness, /actuator/metrics, /actuator/info
- JWT still required on business APIs; health stays public
- Correlation id appears in logs; no Authorization text

SUCCESS CRITERIA (CONTINUED)

- oc whoami works; manifests applied in the assigned project (not 00-namespace.yaml)
- Images pushed; pods Ready; Account Route readiness 200
- **Pipeline:** CRITICAL scan fails the gate; SBOM reviewed; signature command reviewed
- oc rollout undo restored 1.0.0 (history shows more than one revision)
- Ownership map for Exercise 4.3 filled

KEY TAKEAWAY Tick every criterion before moving on -- the next lab builds directly on this one.